

Spring Security + JWT Authentication Flow (Timeline Style)

This document explains the complete authentication flow exactly as it happens at runtime. It follows a strict chronological timeline so that beginners can clearly understand what happens first, next, and why each component exists.

0■■■ Application Startup (Before Any Request)

Spring initializes all security-related beans:

- SecurityConfig
- JwtFilter
- JwtService (secret key generated once)
- CustomUserDetailsService
- AuthenticationManager
- DaoAuthenticationProvider

SecurityFilterChain is built:

- CSRF disabled
- Session policy = STATELESS
- /auth/** → permitAll
- all other endpoints → authenticated
- JwtFilter placed before UsernamePasswordAuthenticationFilter

Application is now ready to accept requests.

1■■■ REGISTER FLOW (POST /auth/register)

Request:

POST /auth/register

Content-Type: application/json

```
{
  "email": "user@gmail.com",
  "password": "password123",
  "role": "USER"
}
```

Flow:

1. Request enters SecurityFilterChain.
2. JwtFilter runs (no Authorization header).
3. Authorization allows /auth/**.
4. AuthController.register() is called.
5. AuthService.register():
 - Password encoded
 - User saved in DB
6. Response returned.

200 OK

User registered successfully

2■■■ LOGIN FLOW (POST /auth/login)

Request:

POST /auth/login

Content-Type: application/json

```
{
  "email": "user@gmail.com",
  "password": "password123"
}
```

Flow:

1. Request enters SecurityFilterChain.
2. JwtFilter runs (no JWT present).
3. Authorization allows /auth/**.
4. AuthController.login() is invoked.
5. AuthenticationManager.authenticate() is called.
6. DaoAuthenticationProvider executes.
7. UserDetailsService loads user from DB.
8. PasswordEncoder verifies password.
9. On success:
 - Authentication object returned
10. JwtService.generateToken() called.
11. JWT returned to client.

200 OK

<JWT TOKEN>

3■■■ PROTECTED API FLOW (GET /api/hello)

Request:

GET /api/hello

Authorization: Bearer <JWT>

Flow:

1. Request enters SecurityFilterChain.
2. JwtFilter extracts JWT.
3. JwtService validates token.
4. UserDetailsService loads user.
5. Authentication stored in SecurityContextHolder.
6. Authorization check passes.
7. Controller executes.

200 OK

Hello

■ Final Mental Model

REGISTER:

Controller → Save user

LOGIN:

Controller

- AuthenticationManager
- AuthenticationProvider
- UserDetailsService
- PasswordEncoder
- JWT returned

EVERY PROTECTED REQUEST:

JwtFilter

- Validate JWT
- Set SecurityContext
- Authorization
- Controller